Plan de pruebas de la aplicación Bicis

Historia de usuario: Detectar la ciudad automáticamente según mi ubicación (ID 532319).

Yo, como visitante temporal, quiero que la aplicación detecte la ciudad en la que estoy a partir de mi ubicación de manera que pueda ver sus estaciones nada más llegar, sin tener que buscar la ciudad en la lista.

Los niveles de prueba que se van a aplicar son los siguientes:

- Pruebas de aceptación. Son los criterios de aceptación acordados con el Product Owner. Se ejecutan de forma manual en el Sprint Review.
- Pruebas de interfaz. Hay una por cada criterio de aceptación. Las que se pueden automatizar se implementan con Espresso, fijando la ubicación del dispositivo con la clase MockLocationManager o con un proveedor de ubicación simulado mediante Hilt.
- Pruebas unitarias. Se aplican a los métodos nuevos o modificados de NetworkLocator y MainPresenter, con técnicas de caja negra (partición equivalente y AVL), usando JUnit. En el presenter la vista, el repositorio y la ubicación se sustituyen por mocks de Mockito.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel.

CRITERIOS DE ACEPTACIÓN

Prueba 1. Caso de éxito

Contexto: el usuario ha dado permiso de ubicación, el móvil tiene la ubicación activada y el usuario está a 20 km o menos de una ciudad con red de bicis (por ejemplo, en Santander).

1. El usuario abre la aplicación.
2. La aplicación carga las redes de bicis.
3. La aplicación obtiene la ubicación del móvil.
4. La aplicación busca la red más cercana a la ubicación del usuario (ver NOTA).
5. La aplicación abre directamente la lista de estaciones de esa red, sin que el usuario tenga que buscarla.
6. Se verifica que aparece en pantalla el nombre de la ciudad detectada (por ejemplo, "Santander").
7. Se verifica que las estaciones que se muestran son las de la red de esa ciudad (por ejemplo, TUeBICI).

NOTA: Para saber la ciudad no se usa el nombre de la ciudad ni ningún servicio externo. Cada red trae la latitud y longitud del centro de su ciudad, así que la aplicación calcula la distancia en línea recta desde el usuario hasta cada red y se queda con la más cercana, siempre que esté a 20 km o menos. Las redes que no tienen coordenadas o las tienen fuera de rango (latitud entre -90 y 90, longitud entre -180 y 180) no se tienen en cuenta.

Prueba 2. Varias redes cerca del usuario

Contexto: hay más de una red a 20 km o menos del usuario (por ejemplo, dos redes en la misma ciudad).

1. El usuario abre la aplicación.
2. La aplicación carga las redes y obtiene la ubicación del móvil.

3. Se verifica que se abre la lista de estaciones de la red que está más cerca del usuario.

4. Se verifica que aparece el nombre de la ciudad de esa red.

Prueba 3. Permiso de ubicación denegado

Contexto: el usuario no da permiso para usar su ubicación, o ya lo denegó otra vez.

1. El usuario abre la aplicación.

2. La aplicación carga las redes de bicis.

3. La aplicación pide permiso de ubicación y el usuario lo deniega.

4. Se verifica que aparece el mensaje "No has dado permiso para usar tu ubicación. Elige una ciudad de la lista".

5. Se verifica que se muestra la lista con todas las ciudades para que el usuario elija una a mano.

Prueba 4. No se puede obtener la ubicación

Contexto: el usuario ha dado permiso, pero no se puede obtener su ubicación (tiene la ubicación del móvil desactivada, no hay señal o tarda demasiado).

1. El usuario abre la aplicación.

2. La aplicación carga las redes de bicis.

3. La aplicación intenta obtener la ubicación del móvil y no lo consigue.

4. Se verifica que aparece el mensaje "No se ha podido obtener tu ubicación. Elige una ciudad de la lista".

5. Se verifica que se muestra la lista con todas las ciudades.

Prueba 5. No hay red de bicis en la ciudad del usuario

Contexto: se obtiene la ubicación sin problema, pero no hay ninguna red a 20 km o menos (por ejemplo, el usuario está en Reinosa).

1. El usuario abre la aplicación.

2. La aplicación carga las redes y obtiene la ubicación del móvil.

3. Se verifica que aparece el mensaje "No hay ninguna red de bicis en tu ciudad. Elige una ciudad de la lista".

4. Se verifica que se muestra la lista con todas las ciudades.

Prueba 6. No se pueden cargar las redes

Contexto: no hay conexión a internet o el servicio no responde.

1. El usuario abre la aplicación.

2. La aplicación intenta cargar las redes y no lo consigue.

3. Se verifica que aparece el mensaje "Ha habido un error cargando las redes".

4. Se verifica que no se pide el permiso de ubicación ni se intenta detectar la ciudad, porque sin redes no se puede.

5. Se verifica que la aplicación no se cierra.

PRUEBAS DE INTERFAZ

Hay una prueba de interfaz por cada criterio de aceptación. Las pruebas automatizadas se ejecutan con Espresso sobre la aplicación con los datos del Anexo 1, concediendo el permiso de ubicación con GrantPermissionRule. La ubicación del dispositivo se fija con la clase MockLocationManager, que sustituye el GPS del dispositivo por uno de prueba, salvo en UI.4, donde se simula con Hilt un proveedor de ubicación que falla. Las pruebas que no se pueden

automatizar quedan cubiertas por las pruebas unitarias indicadas y por la prueba de aceptación manual.

Tabla 1. Pruebas de interfaz

ID	Criterio	Implementación y entrada	Resultado esperado
UI.1	Prueba 1. Caso de éxito	DetectarCiudadExitoUITest. MockLocationManager en Santander (43.4623057, -3.8099803)	Se abren las estaciones de TUeBICI y se muestra "Santander"
UI.2	Prueba 2. Varias redes cerca	Prueba manual: los datos de prueba no tienen dos redes en la misma ciudad. Lógica cubierta por UMP.1f	Se abren las estaciones de la red más cercana
UI.3	Prueba 3. Permiso denegado	Prueba manual: el permiso se pide con un diálogo del sistema. Lógica cubierta por UMP.1b	Mensaje de permiso denegado y lista con las 3 redes
UI.4	Prueba 4. No se puede obtener la ubicación	DetectarCiudadSinUbicacionUI Test. Proveedor de ubicación simulado que falla	Mensaje "No se ha podido obtener tu ubicación..." y lista con las 3 redes
UI.5	Prueba 5. No hay red en la ciudad	DetectarCiudadSinRedUITest. MockLocationManager en Reinosa (43.0018, -4.1376)	Mensaje "No hay ninguna red de bicis en tu ciudad..." y lista con las 3 redes
UI.6	Prueba 6. No se pueden cargar las redes	Prueba manual: la versión de pruebas carga las redes de un fichero local. Lógica cubierta por UMP.1d	Mensaje de error de carga, no se pide el permiso y la app no se cierra

MÉTODOS A PROBAR

Métodos nuevos o modificados para esta historia de usuario, agrupados por clase:

- **Clase NetworkLocator:** findNearestNetwork (nuevo) y distanceKm (nuevo).
- **Clase MainPresenter:** init (modificado: pide el permiso tras cargar las redes) y onLocationPermissionResult (nuevo).
- **Clase DeviceLocationProvider:** requestCurrentLocation (nuevo). Depende de Android, por lo que se prueba con las pruebas de interfaz (UI.1 y UI.5).
- **Clase MainView:** requestLocationPermission, showLocationPermissionDenied, showLocationUnavailable, showNoNetworkNearby y showNetworkStations (nuevos). Son métodos de la vista y se prueban con las pruebas de interfaz.

PRUEBAS UNITARIAS

Pruebas unitarias de la clase NetworkLocator

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba de cada método. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión. Las redes de partida son las del Anexo 1 (con las coordenadas redondeadas a 4 decimales) y el radio máximo es de 20 km salvo que se indique otro valor. Implementadas en NetworkLocatorTest.

- Método findNearestNetwork(networks, lat, lon, maxKm)

Tabla 2. Casos de prueba del método findNearestNetwork

Identificador	Entrada	Valor esperado
UNL.1a	(43.4600, -3.8000) (Usuario en Santander)	TUeBICI
UNL.1b	(40.4618, -3.7038) (A unos 5 km de Madrid)	BiciMAD
UNL.1c	(48.8566, 2.3522) (París, lejos de todas las redes)	null
UNL.1d	(40.4550, -3.7038) con otra red en (40.4500, -3.7038) (Dos redes en la misma ciudad)	La otra red (la más cercana)
UNL.1e	(43.4623, -3.8099) con redes sin location, con coordenadas null, fuera de rango o null (Datos incorrectos)	TUeBICI
UNL.1f	Lista vacía	null
UNL.1g	Lista null	null
UNL.1h	(43.3000, -3.8099), maxKm = distancia a TUeBICI y maxKm - 0,01 (Valor límite del radio)	TUeBICI / null

- Método distanceKm(lat1, lon1, lat2, lon2)

Identificador	Entrada	Valor esperado
UNL.2a	(43.4623, -3.8099), (40.4168, -3.7038) (Santander - Madrid)	338,8 km (± 1 km)
UNL.2b	(40.4168, -3.7038), (40.4168, -3.7038) (Mismo punto)	0 km

Pruebas unitarias del presenter MainPresenter

Se prueba el presenter aislado. La vista (IMainContract.View), el repositorio de redes (INetworksRepository) y el proveedor de ubicación (ILocationProvider) se sustituyen por mocks de Mockito, y se comprueba qué métodos de la vista llama el presenter en cada caso. Las redes de partida son TUeBICI y BiciMAD. Implementadas en MainPresenterTest.

- Métodos init y onLocationPermissionResult

Tabla 3. Casos de prueba del presenter

Identificador	Entrada	Valor esperado
UMP.1a	Permiso concedido, ubicación (43.4623, -3.8099) (Santander)	showNetworkStations(TUeBICI) y ningún mensaje de error
UMP.1b	Permiso denegado	showLocationPermissionDenied( ) y no se pide la ubicación
UMP.1c	Permiso concedido, falla la ubicación	showLocationUnavailable()
UMP.1d	Falla la carga de redes	showLoadError() y no se pide el permiso
UMP.1e	Permiso concedido, ubicación (43.0018, -4.1376) (Reinosa, sin red a 20 km o menos)	showNoNetworkNearby()
UMP.1f	Otra red en (43.4500, -3.8099), ubicación (43.4520, -3.8099) (Dos redes cerca)	showNetworkStations de la red más cercana

ANEXO 1. DATOS DE PRUEBA

Los datos con los que se lanza la aplicación en modo debug (res/raw/networks.json), y las ubicaciones que se simulan en las pruebas, son los que se muestran en las Tablas 4 y 5.

Tabla 4. Redes de bicicletas de prueba

Red	Ciudad	Latitud	Longitud
Bicing	Barcelona, ES	41.3850639	2.1734035
BiciMAD	Madrid, ES	40.4168	-3.7038
TUeBICI	Santander, ES	43.4623057	-3.8099803

Tabla 5. Ubicaciones simuladas

Ubicación	Coordenadas	Se usa en
Santander	(43.4623057, -3.8099803)	UI.1, UMP.1a
Reinosa	(43.0018, -4.1376)	UI.5, UMP.1e
Error de ubicación	-	UI.4, UMP.1c